



ASSIGNMENT-1

SURE TRUST

G3 25 Physical Design VLSI Project

**Submitted to
Submitted by**

Prof. Radha Kumari Mam & Mr. Veeramani Sir
Bhanu Sharma - G3 25 VLSI



Table of Contents / Index

S. No.	Topic	Page No.
1	Introduction	2
2	Objective of the Assignment	3
3	Scope of the Assignment	3
4	Linux Commands Overview	3
5	Basic Linux Commands	4
6	File and Directory Management Commands	5
7	Text Processing Commands - grep and awk	8
8	File Permission Command - chmod	10
9	Vi Editor	10
10	OpenLane Environment Overview	16
11	OpenLane Architecture and Flow	16
12	OpenLane Directory Structure	16
13	OpenLane Environment	17
14	Report and Log Analysis	19
15	Wire Length Analysis	22
16	TCL Script for Wire Length Extraction	22
17	Output of Wire Length Report	25
18	Clock Wire Length and Data Wire Length	26
19	Top 10 Cells Receiving Less Voltage	27
20	IR Drop Analysis	28
21	TCL Script for Worst IR Drop	28
22	Conclusion	35

Introduction

This assignment focuses on understanding basic Linux commands and their practical usage in the OpenLane physical design environment. Linux commands are widely used in VLSI design flows for file handling, directory navigation, text processing, permission management, and automation through scripts.

The assignment also introduces the OpenLane environment, an open-source RTL-to-GDSII physical design flow. OpenLane includes synthesis, floorplanning, placement, clock tree synthesis, routing, static timing analysis, and physical verification. These stages convert an RTL design into a final chip layout.

In this assignment, various Linux commands were executed and verified through terminal screenshots. Further, OpenLane directory structure, reports, logs, wire length analysis, IR drop analysis, and TCL scripting were explored to understand the physical design flow in a practical manner.

Objective of the Assignment

The main objective of this assignment is to understand the use of Linux commands and their importance in the VLSI physical design flow. It also aims to provide practical knowledge of OpenLane flow, report analysis, and TCL-based automation.

S. No.	Objective
1	Understand basic Linux commands used in terminal-based design environments
2	Perform file and directory operations using Linux commands
3	Learn text-processing commands such as grep and awk
4	Understand file permission management using the chmod command
5	Understand the use of the vi editor in Linux



6	Explore the OpenLane physical design environment
7	Understand OpenLane architecture and directory structure
8	Analyze reports and logs generated during the physical design flow
9	Extract wire length information using TCL scripting
10	Study IR drop and voltage degradation analysis in physical design

Scope of the Assignment

The scope of this assignment includes basic Linux command execution, OpenLane environment exploration, and physical design report analysis. It covers both theoretical explanation and practical screenshots to show command execution and output verification.

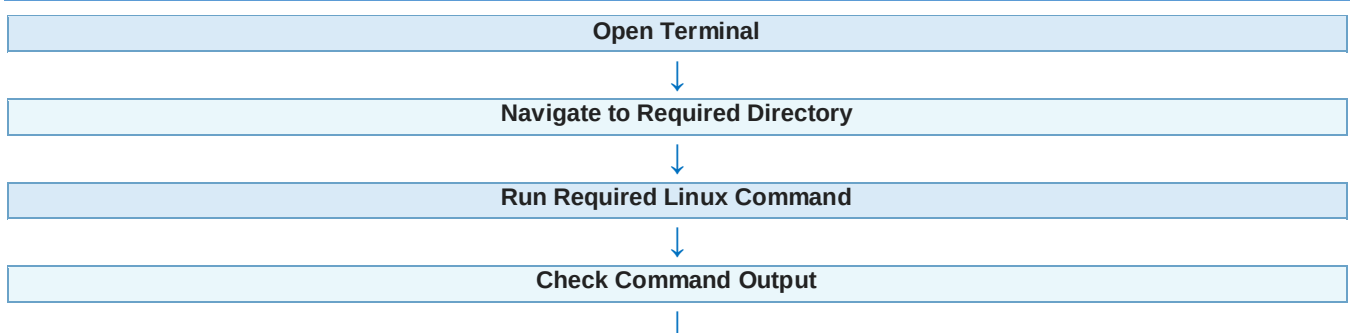
Area	Covered Topics
Linux Basics	ls, pwd, mkdir, cd, rmdir, cat, cp, mv, rm, uname, touch, ps, echo, wget, sort
Text Processing	grep, awk
File Permission	chmod command
Text Editing	vi editor commands and modes
OpenLane Flow	Architecture, directory structure, reports, logs
Report Analysis	Timing, routing, wire length, IR drop
Automation	TCL scripts for extracting design metrics

Linux Commands Overview

Linux commands are used to interact with the operating system through the terminal. In VLSI physical design, most tools are executed in a Linux environment, so basic command-line knowledge is very important. These commands help create files, manage directories, view file contents, search text patterns, change permissions, and run scripts.

Command	Use
ls	Lists files and directories
pwd	Shows current directory path
mkdir	Creates a new directory
cd	Changes current directory
rmdir	Removes an empty directory
cat	Displays file content
cp	Copies files or directories
mv	Moves or renames files
rm	Deletes files
grep	Searches text patterns
awk	Processes and extracts text data
chmod	Changes file permissions

Linux Command Usage Flowchart





Basic Linux Commands

Ls command :

This will show the names of files and folders in your current directory.

List with Details (-l)

List Human-Readable Sizes (-lh)

List All Files, Including Hidden (-a)

Sort by Time (-t) and Reverse (-r)

```
Apr 27 22:25
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane

bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ ls
Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls
Openlane  README.md  tool_install2.sh  tool_install.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls -l
total 16
drwxrwxr-x 14 bhanu-sharma bhanu-sharma 4096 Apr 19 17:21 Openlane
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  10 Apr 19 16:51 README.md
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  573 Apr 19 16:51 tool_install2.sh
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 1377 Apr 19 16:51 tool_install.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls -lh
total 16K
drwxrwxr-x 14 bhanu-sharma bhanu-sharma 4.0K Apr 19 17:21 Openlane
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  10 Apr 19 16:51 README.md
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  573 Apr 19 16:51 tool_install2.sh
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 1.4K Apr 19 16:51 tool_install.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls -a
.  ..  .git  Openlane  README.md  tool_install2.sh  tool_install.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls -lt
total 16
drwxrwxr-x 14 bhanu-sharma bhanu-sharma 4096 Apr 19 17:21 Openlane
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  10 Apr 19 16:51 README.md
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  573 Apr 19 16:51 tool_install2.sh
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 1377 Apr 19 16:51 tool_install.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls -ltr
total 16
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 1377 Apr 19 16:51 tool_install.sh
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  573 Apr 19 16:51 tool_install2.sh
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma  10 Apr 19 16:51 README.md
drwxrwxr-x 14 bhanu-sharma bhanu-sharma 4096 Apr 19 17:21 Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$
```

Pwd command :

The pwd command in linux is used to display the full path of the current directory.



```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ pwd
/home/bhanu-sharma/Desktop/Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$
```

3. mkdir command :

The mkdir command in linux is used to create new directories or folders.

```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ mkdir vlsi_project
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls
OpenLane  README.md  tool_install2.sh  tool_install.sh  vlsi_project
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$
```

4. Cd command :

The cd command in linux is used to change the current working directory.

allows navigation between directories.

moves the user to the home directory when used without arguments.



```
Apr 28 22:14
bhanu-sharma@bhanu-sharma-VirtualBox: ~
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ cd vlsi_project
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/vlsi_project$ cd
bhanu-sharma@bhanu-sharma-VirtualBox:~$ S
```

5. 5. rmdir

The rmdir command in linux is used to delete empty directories.

```
Apr 28 22:20
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ rmdir vlsi_project
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls
OpenLane  README.md  tool_install2.sh  tool_install.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$
```

6. Cat Command :

The cat command in linux is used to display file contents and combine multiple files.

```
Apr 28 22:51
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ cat Ubuntu.txt
Hello Ubuntu !
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```

7. Cp Command :

The cp command in linux is used to copy files or directories.

```
Apr 28 23:15
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ cat Ubuntu.txt
Hello Ubuntu !
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ cat Ubuntu-Vlsi.txt
Hi VLSI!
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ cp Ubuntu.txt Ubuntu-Vlsi.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ cat Ubuntu-Vlsi.txt
Hello Ubuntu !
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```



8. mv Command :

The mv command in Linux is used to move or rename files and directories.

```
Apr 28 23:27
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ ls
Ubuntu.txt  Ubuntu-Vlsi.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ mv Ubuntu.txt Ubuntu-linux.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ ls
Ubuntu-linux.txt  Ubuntu-Vlsi.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```

9. rm

The rm command in Linux is used to delete files permanently.

```
Apr 28 23:46
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ ls
Ubuntu-linux.txt  Ubuntu-Vlsi.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ rm Ubuntu-linux.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ ls
Ubuntu-Vlsi.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```

10. uname

The uname command in Linux is used to display system information.

```
Apr 28 23:53
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ uname
Linux
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ SS
```

11. Touch Command :

The touch command in linux is used to create empty files and updates file timestamps.



```
Apr 30 00:25
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ ls
Ubuntu-Vlsi.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ touch Openlane-project.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ ls
Openlane-project.txt  Ubuntu-Vlsi.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```

12. Ps Command :

The ps command in linux is used to display running processes.

```
Apr 30 21:39
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ ps
  PID TTY          TIME CMD
 3489 pts/0    00:00:00 bash
 3594 pts/0    00:00:00 ps
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```

13. Grep Commands :

The grep command in Linux is used to search text patterns.

Finds specific strings.

Filters output.

```
Apr 30 22:07
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ grep "Catholic" Ubuntu-Vlsi.txt
Hello Ubuntu ! Nory was a Catholic because her mother was a Catholic, and Nory's mother was a Catholic because her father was a Catholic, and her father was a Catholic because his mother was a Catholic, or had been.
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```

14. Echo Command :

The echo command in Linux is used to display text in the terminal.

```
Apr 30 22:11
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ echo "Hello Sure Trust from Ubuntu "
Hello Sure Trust from Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$
```

15. Wget Command :

The wget command in Linux is used to download files from the internet using URL



```
Apr 30 22:56
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ wget http://example.com/file.zip
--2026-04-30 22:56:18-- http://example.com/file.zip
Resolving example.com (example.com)... 172.66.147.243, 104.20.23.154, 2606:4700:9645:72db:f264:1:ef6b:ff98
Connecting to example.com (example.com)|172.66.147.243|:80... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-04-30 22:56:18 ERROR 404: Not Found.
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$
```

21. Sort Commands :

The sort command in Linux is used to sort file contents.

```
Apr 30 23:11
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ cd Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$ sort Ubuntu-Vlsi.txt
1
10
2
3
4
5
6
7
8
9
Hello Ubuntu !
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Ubuntu$
```



CHmod Command

chmod Command in Linux

The chmod command in Linux is used to change the access permissions of files and directories. This command is essential for managing who can read, write, or execute a file or directory. Permissions are divided into three categories: read (r), write (w), and execute (x), and they apply to three classes of users: the owner (u), the group (g), and others (o).

Syntax and Usage

The basic syntax of the chmod command is:

`chmod [OPTIONS] MODE FILE`

OPTIONS: Optional flags that modify the behavior of the command.

MODE: The permissions to be set, represented either by a three-digit octal number or symbolic notation.

FILE: The name of the file or directory for which the permissions are to be changed.

```
May 3 13:25
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ vi Ubuntu.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ chmod +x Ubuntu.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ ./Ubuntu.sh
Hello Ubuntu
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$
```

Vi editor :

The vi editor is the default text editor in most UNIX/Linux systems. It is a modal editor, operating mainly in Command Mode, Insert Mode, and Last Line Mode. Mastering its commands allows for fast and efficient text editing directly from the terminal.

Opening and Closing Files

- `vi filename` → Open or create a file.
- `:w` → Save changes.
- `:wq` or `:x` → Save and exit.
- `:q` → Quit without saving.
- `:q!` → Force quit without saving.



Modes in vi

Command Mode (default): Navigate, delete, copy, and execute commands.

Insert Mode: Press i, a, o, A, or O to insert text. Exit with Esc.

Last Line Mode: Press : in Command Mode to run commands like :wq or :s/old/new/g.

Navigation

h / l → Move left/right.

j / k → Move down/up.

0 / \$ → Start/end of line.

w / b → Next/previous word.

gg → Start of file.

G → End of file.

CTRL+f / CTRL+b → Page forward/backward.

Editing

dd → Delete current line.

yy → Yank (copy) current line.

p / P → Paste after/before cursor.

u → Undo last change.

r → Replace single character.

x → Delete character under cursor.

Search and Replace

/pattern → Search forward.

?pattern → Search backward.

n / N → Repeat search forward/backward.

:s/old/new/ → Replace first occurrence in line.

:%s/old/new/g → Replace all occurrences in file.

Example Workflow

```
vi notes.txt # Open file
```

```
i # Enter insert mode
```

Type your text here

```
Esc # Back to command mode
```

```
:wq # Save and exit
```

Grep Command :

The grep command in Linux/Unix is a powerful text-search utility used to find lines matching a given pattern in files or input streams. The name stands for Global Regular Expression Print, and it supports both simple strings and complex regular expressions for pattern matching.

Basic Syntax:

grep [options] pattern [file...]

pattern: String or regex to search.

file: One or more files to search in.

```
# Search for "python" in notes.txt
```

```
grep "python" notes.txt
```

```
# Case-insensitive search
```

```
grep -i "unix" geekfile.txt
```

```
# Search recursively in directories
```

```
grep -R "httpd" /etc/
```

```
# Match whole words only
```

```
grep -w "error" logfile.txt
```

```
# Show line numbers
```



```
grep -n "main" program.c
# Count matching lines
grep -c "failed" auth.log
# Invert match (show lines without the pattern)
grep -v "success" output.txt
```

```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/logs/synthesis$ grep -i error 1-synthesis.log
ABC: Error: The network is combinational.
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/logs/synthesis$ grep -in error 1-synthesis.log
686:ABC: Error: The network is combinational.
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/logs/synthesis$ grep -in warning 1-synthesis.log
665:ABC: Warning: Detected 2 multi-output gates (for example, "sky130_fd_sc_hd__fa_1").
777:Warning: Wire spm.\p is used but has no driver.
808:Warnings: 1 unique messages, 1 total
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/logs/synthesis$
```

5. Awk command :

The awk command is a powerful text-processing tool in Linux, designed for pattern scanning and processing. It allows users to manipulate data, extract specific fields, and generate reports efficiently. Below is an overview of its usage, syntax, and practical examples.

Syntax

awk [options] 'pattern {action}' input-file

Options: Flags like -F to set a field separator or -f to read an awk script from a file.

Pattern: Specifies the condition to match lines.

Action: Defines what to do with the matched lines.

Common Use Cases and Examples

Print All Lines

By default, awk prints every line of the input file:

```
awk '{print}' employee.txt
```

Search for a Pattern

To print lines containing the word "manager":

```
awk '/manager/ {print}' employee.txt
```

Print Specific Columns

Extract specific fields (e.g., Name and Salary from columns 1 and 4):

```
awk '{print $1, $4}' employee.txt
```

Built-in Variables

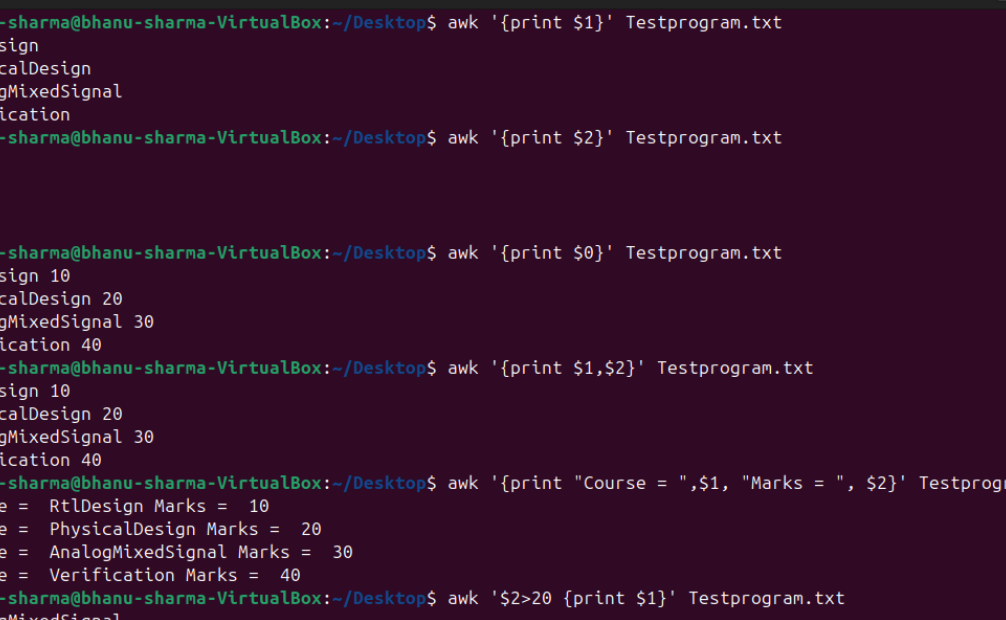
NR: Line number of the current record.

NF: Number of fields in the current record.

FS: Input field separator (default is whitespace).

OFS: Output field separator (default is a space).

Page 13



```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ awk '{print $1}' Testprogram.txt
RtlDesign
PhysicalDesign
AnalogMixedSignal
Verification
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ awk '{print $2}' Testprogram.txt
10
20
30
40
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ awk '{print $0}' Testprogram.txt
RtlDesign 10
PhysicalDesign 20
AnalogMixedSignal 30
Verification 40
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ awk '{print $1,$2}' Testprogram.txt
RtlDesign 10
PhysicalDesign 20
AnalogMixedSignal 30
Verification 40
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ awk '{print "Course = ",$1, "Marks = ", $2}' Testprogram.txt
Course = RtlDesign Marks = 10
Course = PhysicalDesign Marks = 20
Course = AnalogMixedSignal Marks = 30
Course = Verification Marks = 40
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ awk '$2>20 {print $1}' Testprogram.txt
AnalogMixedSignal
Verification
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$
```

May 3 15:12

bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop

```
RtlDesign,10
PhysicalDesign,20
AnalogMixedSignal,30
Verification,40
```



```
May 3 15:21
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop

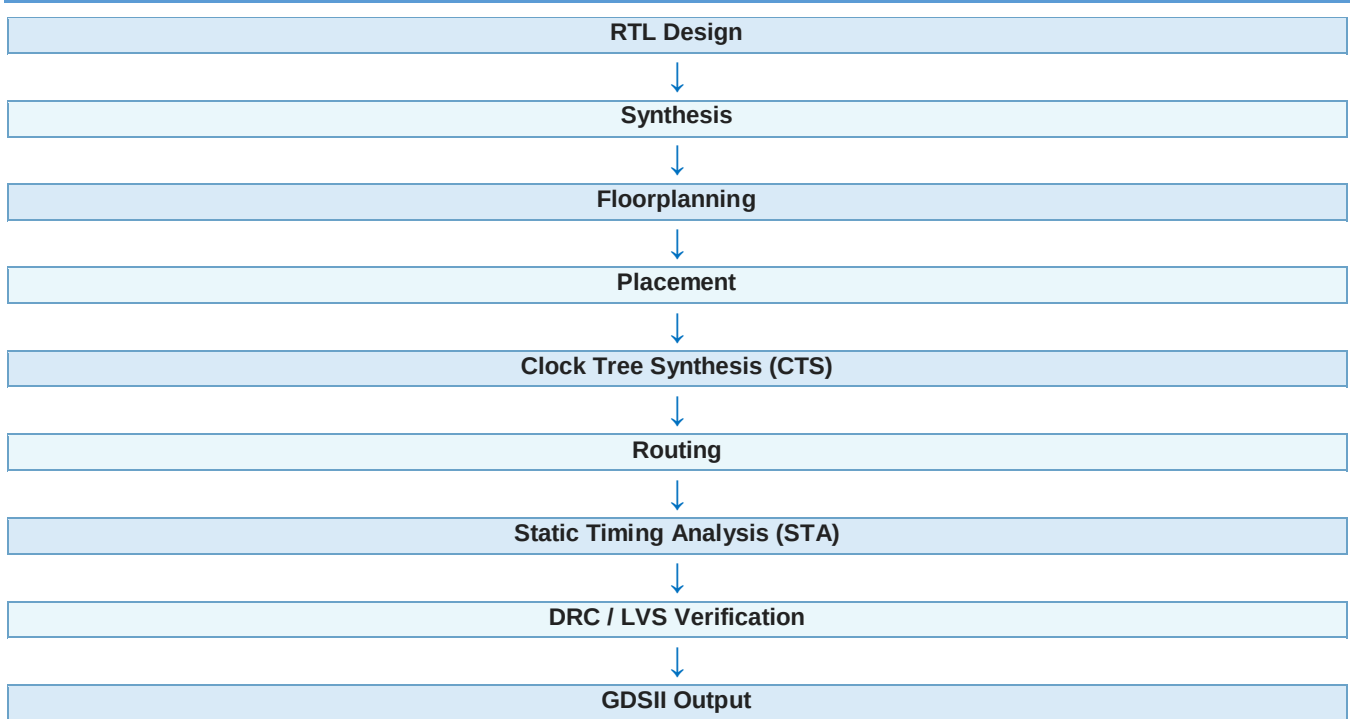
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ vi Testprogram.txt
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ awk -F "," '{print $1}' Testprogram.txt
RtlDesign
PhysicalDesign
AnalogMixedSignal
Verification
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ ls -l | awk '{print $1, $9}'
total
drwxrwxr-x Openlane
-rw-rw-r-- Testprogram.txt
drwxrwxr-x Ubuntu
-rwxrwxrwx Ubuntu-copy.txt
-rw-rw-r-- Ubuntu-Project.txt
-rwxrwxr-x Ubuntu.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$ ls -l
total 24
drwxrwxr-x 4 bhanu-sharma bhanu-sharma 4096 Apr 28 22:32 Openlane
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 69 May 3 15:16 Testprogram.txt
drwxrwxr-x 2 bhanu-sharma bhanu-sharma 4096 May 2 22:14 Ubuntu
-rwxrwxrwx 1 bhanu-sharma bhanu-sharma 116 May 3 10:34 Ubuntu-copy.txt
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 45 May 3 09:48 Ubuntu-Project.txt
-rwxrwxr-x 1 bhanu-sharma bhanu-sharma 35 May 3 13:02 Ubuntu.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop$
```



OpenLane Environment Overview

OpenLane is an open-source automated physical design flow used for converting RTL design into GDSII layout. It combines several open-source EDA tools and supports the main ASIC implementation stages from synthesis to final physical verification.

Physical Design Flowchart



Stage	Purpose	Main Output
RTL Synthesis	Converts Verilog RTL into a gate-level netlist	Synthesized netlist
Floorplanning	Defines chip/core area and pin placement	Floorplan DEF
Placement	Places standard cells inside the core area	Placed DEF
CTS	Builds the clock distribution network	Clock tree reports
Routing	Connects all nets using metal layers	Routed DEF
STA	Checks setup and hold timing	Timing reports
DRC/LVS	Verifies physical layout correctness	Verification reports
GDS Generation	Generates final chip layout	GDSII file

OpenLane Directory Structure

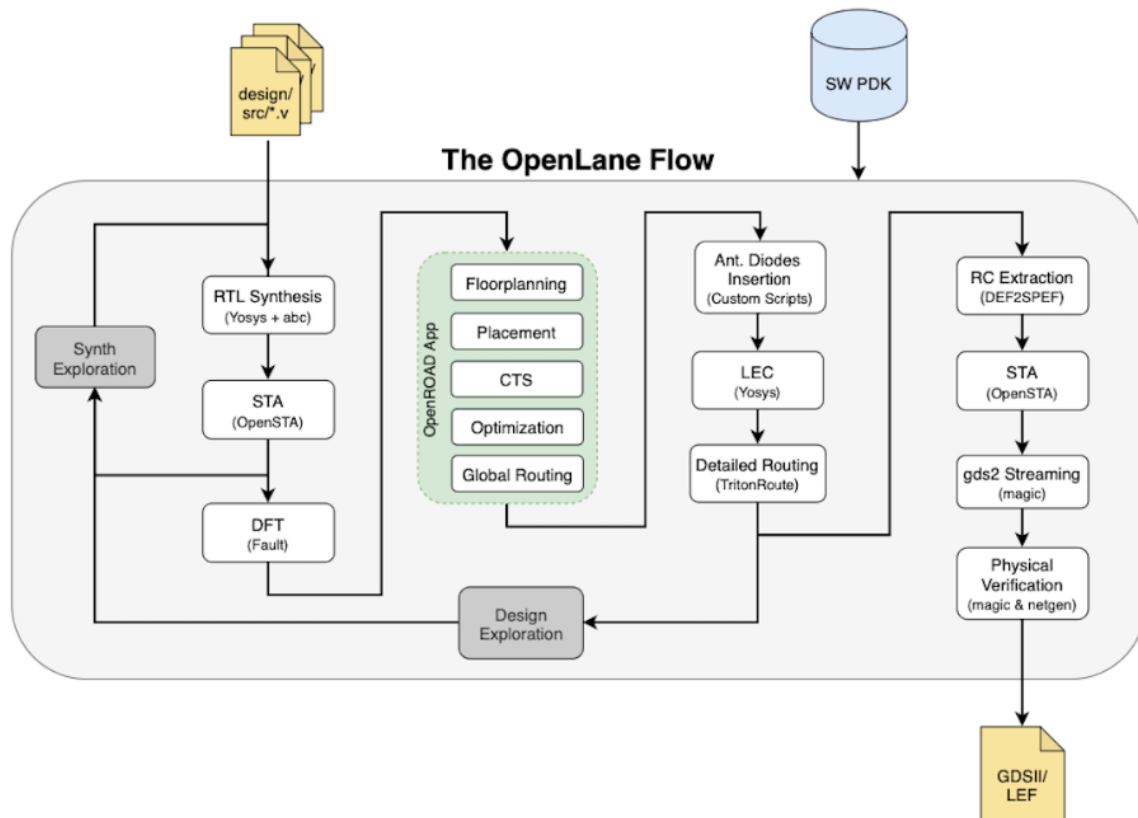
Directory / File	Description
designs/	Contains design files for each project
config.tcl	Main configuration file for OpenLane flow
runs/	Stores output files generated after running OpenLane
logs/	Contains log files for each design stage
reports/	Contains timing, area, power, and DRC reports
results/	Stores final DEF, GDS, netlist, and routed files
src/	Contains RTL Verilog source files
pdks/	Contains process design kit files



OpenLane Environment :



OpenLane Architecture



```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane$ tree
.
├── OpenLane
│   ├── AUTHORS.md
│   ├── configuration
│   │   ├── checkers.tcl
│   │   ├── cts.tcl
│   │   ├── extraction.tcl
│   │   ├── floorplan.tcl
│   │   ├── general.tcl
│   │   ├── load_order.txt
│   │   ├── placement.tcl
│   │   ├── routing.tcl
│   │   └── synthesis.tcl
│   ├── CONTRIBUTING.md
│   ├── default.cvcrc
│   ├── dependencies
│   │   ├── arch
│   │   │   └── run_time.txt
│   │   ├── centos-7
│   │   │   ├── compile_time.txt
│   │   │   ├── precompile_time.txt
│   │   │   └── run_time.txt
│   └── env_info.py
```



```
May 3 15:39
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ cd Openlane
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls -l
total 16
drwxrwxr-x 14 bhanu-sharma bhanu-sharma 4096 Apr 19 17:21 OpenLane
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 10 Apr 19 16:51 README.md
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 573 Apr 19 16:51 tool_install2.sh
-rw-rw-r-- 1 bhanu-sharma bhanu-sharma 1377 Apr 19 16:51 tool_install.sh
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane$ ls -R
.:
OpenLane README.md tool_install2.sh tool_install.sh

./OpenLane:
AUTHORS.md      default.cvcrc  docker  flow.tcl  Jenkinsfile  Makefile      requirements.txt  tests
configuration  dependencies  docs    gui.py    layoutrc     README.md     run_designs.py   venv
CONTRIBUTING.md designs        env.py   install   LICENSE      regression_results  scripts

./OpenLane/configuration:
checkers.tcl  extraction.tcl  general.tcl  placement.tcl  synthesis.tcl
cts.tcl       floorplan.tcl  load_order.txt  routing.tcl

./OpenLane/dependencies:
arch      get_tag.py  includedyaml  python  ubuntu-20.04  version.py
centos-7  hash_for.py  macos        tool_metadata.yml  update_pyyaml.sh
env_info.py  image_name.mk  __pycache__  tool.py        verify_versions.py

./OpenLane/dependencies/arch:
run_time.txt
```

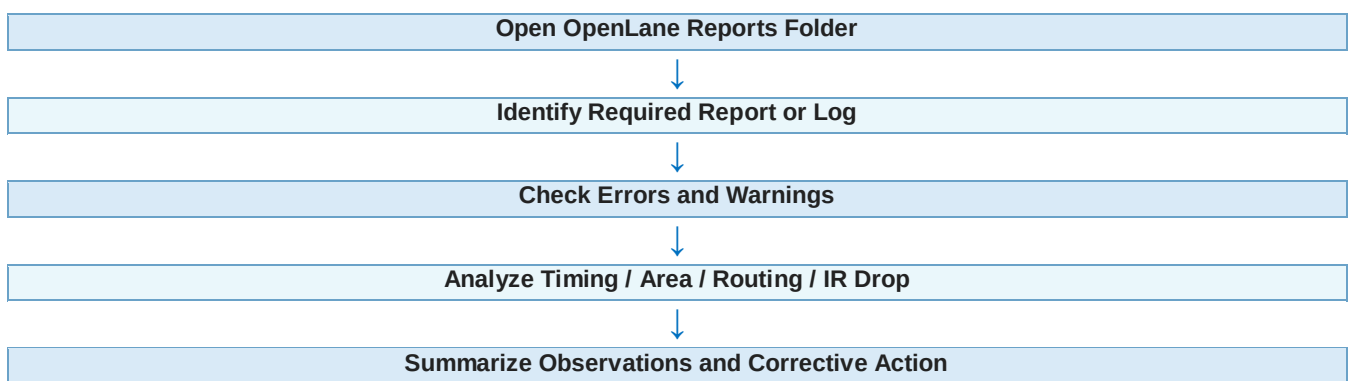


Report and Log Analysis Overview

Report and log analysis is an important step in physical design because it helps verify whether each stage has completed successfully. Reports provide information about timing, area, power, routing, and design-rule violations. Logs help identify warnings, errors, and failed commands during the OpenLane flow.

Report / Log Type	What It Shows	Why It Is Important
Synthesis Report	Area, cell count, gate-level netlist details	Helps check logic optimization
STA Report	Setup/hold timing and slack values	Confirms timing closure
Routing Log	Wire length, metal usage, routing completion	Verifies connectivity
DRC Report	Design rule violations	Required for manufacturability
LVS Report	Layout vs schematic comparison	Ensures layout matches design
Power Report	Internal, switching, and leakage power	Helps analyze power consumption
IR Drop Report	Voltage drop across the power network	Checks power integrity

Report Analysis Flowchart



Report and log analysis :

```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/logs/synthesis$ cat 1-synthesis.log

-----
yosys -- Yosys Open SYnthesis Suite

Copyright (C) 2012 - 2020 Claire Xenia Wolf <claire@yosyshq.com>

Permission to use, copy, modify, and/or distribute this software for any
purpose with or without fee is hereby granted, provided that the above
copyright notice and this permission notice appear in all copies.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES
WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF
MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR
ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN
ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF
OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
-----

Yosys 0.30+48 (git sha1 14d50a176d5, gcc 8.3.1 -fPIC -Os)

[TCL: yosys -import] Command name collision: found pre-existing command 'cd' -> skip.
[TCL: yosys -import] Command name collision: found pre-existing command 'eval' -> skip.
[TCL: yosys -import] Command name collision: found pre-existing command 'exec' -> skip.
[TCL: yosys -import] Command name collision: found pre-existing command 'read' -> skip.
[TCL: yosys -import] Command name collision: found pre-existing command 'trace' -> skip.
```



```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/reports/synthesis
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/reports/synthesis$ ls
1-synthesis.AREA_0.chk.rpt      2-syn_sta.max.rpt
1-synthesis.AREA_0.stat.rpt    2-syn_sta.min.rpt
1-synthesis_dff.stat          2-syn_sta.power.rpt
1-synthesis_pre.stat          2-syn_sta.skew.rpt
1-synthesis_pre_synth.chk.rpt 2-syn_sta.summary.rpt
2-syn_sta.checks.rpt
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/openlane_test/reports/synthesis$ cat 2-syn_sta.summary.rpt
=====
report_tns
=====
tns 0.00

=====
report_wns
=====
wns 0.00

=====
report_worst_slack -max (Setup)
=====
worst slack 6.39

=====
report_worst_slack -min (Hold)
=====

61. Printing statistics.
=== spm ===
Number of wires:          305
Number of wire bits:      336
Number of public wires:   99
Number of public wire bits: 130
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          301
sky130_fd_sc_hd__a21o_2    1
sky130_fd_sc_hd__a31o_2   31
sky130_fd_sc_hd__and2_2    32
sky130_fd_sc_hd__buf_1     15
sky130_fd_sc_hd__dfirtp_2  64
sky130_fd_sc_hd__inv_2     64
sky130_fd_sc_hd__nand2_2   31
sky130_fd_sc_hd__nand3_2   1
sky130_fd_sc_hd__xnor2_2   31
sky130_fd_sc_hd__xor2_2    31

Chip area for module '\spm': 3711.059200

62. Executing Verilog backend.
Dumping module '\spm'.
```



Wire Length Analysis Overview

Wire length analysis is used to understand how much interconnect is required to route the design. A high wire length can increase delay, capacitance, routing congestion, and power consumption. Layer-wise wire length also helps identify which metal layers are used most during routing.

Parameter	Meaning
Total Wire Length	Complete routed interconnect length of the design
MET1 to MET5 Wire Length	Layer-wise distribution of routing wires
Clock Wire Length	Wire length used by the clock network
Data Wire Length	Wire length used by signal/data nets
Routing Congestion	Difficulty in placing/routing nets due to limited routing resources

Total wire length of the design :

Steps to Find Total Wire Length (Using 23-detailed.log)

Step 1

Open terminal.

Step 2

Navigate to the OpenLane directory:

```
cd ~/Desktop/Openlane/OpenLane
```

Step 3

Search for the total wire length in the routing log:

```
grep -R "Total wire length" /home/bhanu-sharma/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/logs/routing
```

```
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane$ grep -R "Total wire length" /home/bhanu-sharma/Desktop/Openlane/Openlane/designs/spm/runs/RUN_1/logs/routing/23-detailed.log:Total wire length = 6785 um.
```

Total Length design using a TCL script :

```
# =====
# Tcl script to extract wire length from OpenLane routing logs
# No separate wire_length_verbos.rpt required
# =====
puts "=====
puts "WIRE LENGTH EXTRACTION FROM ROUTING LOGS"
puts "=====
# -----
# Step 1 : Routing log directory
# -----
set log_dir "designs/spm/runs/openlane_test/logs/routing"
puts "Searching routing logs inside:"
puts $log_dir
# -----
# Step 2 : Find detailed routing log automatically
# -----
set detail_files [glob -nocomplain $log_dir/*detailed*.log]
if {[llength $detail_files] == 0} {
puts "ERROR : No detailed routing log found"
puts "Check routing log directory path"
}
exit 1
```



```
}  
# Select first matching file  
set detail_log [lindex $detail_files 0]  
puts "Routing log found:"  
puts $detail_log  
# -----  
# Step 3 : Output result file  
# -----  
set output_file "final_wire_result.txt"  
puts "Output will be saved in:"  
puts $output_file  
# -----  
# Step 4 : Initialize variables  
# -----  
set total_wl 0  
set met1_wl 0  
set met2_wl 0  
set met3_wl 0  
set met4_wl 0  
set met5_wl 0  
# -----  
# Step 5 : Open files  
# -----  
set fp_in [open $detail_log r]  
set fp_out [open $output_file w]  
# -----  
# Step 6 : Write header in output file  
# -----  
puts $fp_out "=====  
puts $fp_out "FINAL WIRE LENGTH REPORT"  
puts $fp_out "=====  
# -----  
# Step 7 : Read routing log line-by-line  
# -----  
while {[gets $fp_in line] >= 0} {  
# Debug print  
puts "Checking line: $line"  
  
if {[regexp {Total wire length = ([0-9]+)} $line match value]} {  
set total_wl $value  
puts "-----"  
puts "TOTAL WIRE LENGTH FOUND"  
puts "Value = $total_wl um"  
puts "-----"  
puts $fp_out "Total wire length = $total_wl um"  
}  
}  
}  
# -----  
# MET2 wire length  
# -----  
if {[regexp -nocase {LAYER met2 = ([0-9]+)} $line match value]} {  
set met2_wl $value  
puts "MET2 wire length = $met2_wl um"  
puts $fp_out "MET2 wire length = $met2_wl um"  
}  
# -----  
# MET3 wire length  
# -----  
if {[regexp -nocase {LAYER met3 = ([0-9]+)} $line match value]} {  
set met3_wl $value  
puts "MET3 wire length = $met3_wl um"  
puts $fp_out "MET3 wire length = $met3_wl um"  
}  
# -----  
# MET4 wire length  
# -----
```



```
if {[regex -nocase {LAYER met4 = ([0-9]+)} $line match value]} {
  set met4_wl $value
  puts "MET4 wire length = $met4_wl um"
  puts $fp_out "MET4 wire length = $met4_wl um"
}

# -----
# MET5 wire length
# -----
if {[regex -nocase {LAYER met5 = ([0-9]+)} $line match value]} {
  set met5_wl $value
  puts "MET5 wire length = $met5_wl um"
  puts $fp_out "MET5 wire length = $met5_wl um"
}

# -----
# Step 8 : Final summary
# -----
puts ""
puts "=====
puts "FINAL SUMMARY"
puts "=====
puts "Total wire length = $total_wl um"
puts "MET1 wire length = $met1_wl um"
puts "MET2 wire length = $met2_wl um"
puts "MET3 wire length = $met3_wl um"
puts "MET4 wire length = $met4_wl um"
puts "MET5 wire length = $met5_wl um"
puts "=====
puts $fp_out "=====
puts $fp_out "Extraction Completed"
puts $fp_out "=====
# -----
# Step 9 : Close files
# -----
close $fp_in
close $fp_out
puts ""
puts "Result saved in file:"
puts $output_file
```

FINAL SUMMARY

```
=====
Total wire length = 6719 um
MET1 wire length = 2773 um
MET2 wire length = 3819 um
MET3 wire length = 106 um
MET4 wire length = 21 um
MET5 wire length = 0 um
=====
```

Output :

=====

FINAL WIRE LENGTH REPORT

=====

Total wire length = 6785 um
MET1 wire length = 2827 um
MET2 wire length = 3803 um
MET3 wire length = 96 um



MET4 wire length = 58 um
MET5 wire length = 0 um
Total wire length = 6684 um
MET1 wire length = 2742 um
MET2 wire length = 3803 um
MET3 wire length = 101 um
MET4 wire length = 36 um
MET5 wire length = 0 um
Total wire length = 6713 um
MET1 wire length = 2780 um
MET2 wire length = 3808 um
MET3 wire length = 103 um
MET4 wire length = 21 um
MET5 wire length = 0 um
Total wire length = 6719 um
MET1 wire length = 2773 um
MET2 wire length = 3819 um
MET3 wire length = 106 um
MET4 wire length = 21 um
MET5 wire length = 0 um
Total wire length = 6719 um
MET1 wire length = 2773 um
MET2 wire length = 3819 um
MET3 wire length = 106 um
MET4 wire length = 21 um
MET5 wire length = 0 um

=====
Extraction Completed
=====

Note : Multiple wire-length values were observed for MET4 and MET5 because the routing tool performs several routing and optimization iterations. At each iteration, some nets are rerouted to fix congestion and DRC violations, causing the metal-layer wire lengths to change. These values represent intermediate routing states rather than different metal layers.

Key Metrics Summary Table

Parameter	Value	Observation
Total Wire Length	6719 um	Total routed interconnect length
MET1 Wire Length	2773 um	Used for lower-level routing
MET2 Wire Length	3819 um	Highest metal usage in this design
MET3 Wire Length	106 um	Very low usage
MET4 Wire Length	21 um	Minimal usage
MET5 Wire Length	0 um	Not used in final routing

Observation: From the wire length report, MET2 has the highest wire length, which means most of the routing is completed on MET2. MET5 is not used, indicating that the design is simple and does not require higher metal layers.



Clock and Data Wire Length Calculation Manually:

Steps to Find Clock Wire Length (Using wire_lengths.csv)

Step 1

Move to the routing report directory:

```
cd ~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/routing
```

Step 2

Extract all clock-related nets:

```
grep clk 24-wire_lengths.csv
```

Step 3

Note the lengths of all clock nets and add them.

```
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/routing$ grep clk 24-wire_lengths.csv
clknet_0_clk,145.53
clknet_3_7__leaf_clk,105.42
clknet_3_2__leaf_clk,96.52
clknet_3_5__leaf_clk,74.86
clknet_3_0__leaf_clk,74.52
clknet_3_4__leaf_clk,65.18
clknet_3_1__leaf_clk,62.72
clknet_3_3__leaf_clk,58
clknet_3_6__leaf_clk,53.82
```

Clock Wire Length Calculation :

$$145.53 + 105.42 + 96.52 + 74.86 + 74.52 + 65.18 + 62.72 + 58 + 53.82 = 736.57 \text{ um}$$

Total Wire Length

From 23-detailed.log:

Total wire length = 6785 um

Data Wire Length :

Data Wire Length = Total Wire Length – Clock Wire Length

$$= 6785 - 736.57$$
$$= 6048.43 \text{ um}$$

Parameter	Value
Total Wire Length	6785 um
Clock Wire Length	736.57 um



Parameter	Value
Data Wire Length	6048.43 um

Total Wire Length, Clock and Data Wire Length Calculation using TCL Scripting :

```
# =====
# clock_data_length.tcl
# Extract Clock Length and Data Length
# =====

# File paths
set wire_file "/home/bhanu-sharma/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/routing/24-wire_lengths.csv"

set log_file "/home/bhanu-sharma/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/logs/routing/23-detailed.log"

# -----
# Step 1 : Calculate Clock Length
# -----

set fp [open $wire_file r]

set clock_length 0.0

while {[gets $fp line] >= 0} {

    if {[regexp {clk} $line]} {

        set fields [split $line ","]

        set length [lindex $fields 1]

        set clock_length [expr {$clock_length + $length}]
    }
}

close $fp

# -----
# Step 2 : Extract Total Wire Length
# -----

set fp [open $log_file r]

set total_length 0.0

while {[gets $fp line] >= 0} {
```



```
if {[regexp {Total wire length *= *([0-9.]+)} $line -> value]} {  
    set total_length $value  
}  
}  
  
close $fp  
  
# -----  
# Step 3 : Calculate Data Length  
# -----  
  
set data_length [expr {$total_length - $clock_length}]  
  
# -----  
# Step 4 : Report  
# -----  
  
puts ""  
puts "=====  
puts "WIRE LENGTH REPORT"  
puts "=====  
  
puts [format "Total Wire Length : %.2f um" $total_length]  
puts [format "Clock Wire Length : %.2f um" $clock_length]  
puts [format "Data Wire Length : %.2f um" $data_length]  
  
puts "=====
```

Output :

```
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane$ tclsh clock_length.tcl  
  
=====  
WIRE LENGTH REPORT  
=====  
Total Wire Length : 6719.00 um  
Clock Wire Length : 784.48 um  
Data Wire Length : 5934.52 um  
=====
```

Worst IR Dop and % Degradation :

Go to the “designs/spm/runs/RUN_1/reports/signoff/” directory
Find the “32-irdrop-VDD.rpt
32-irdrop-VGND.rpt”

$$\text{IR Drop} = V_{\text{source}} - V_{\text{received}}$$
$$\% \text{Degradation} = (\text{Supply Voltage} / \text{IR Drop}) \times 100$$



```
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ awk -F',' 'NR>
{if($4>max){max=$4; line=$0}} END {print line}' 32-irdrop-VPWR.rpt
_337_, 84.81, 73.44, 1.79977
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ awk -F',' 'NR>
{if($4>max){max=$4; line=$0}} END {print line}' 32-irdrop-VGND.rpt
_453_, 5.52, 59.84, 0.00048724
```

For VDD :

Supply Voltage = 1.8V

$$IR \text{ Drop} = 1.8 - 1.79977 \\ = 0.00023V$$

$$\% \text{ Degradation} = (0.0023/1.8) \times 100 \\ = 0.0128\%$$

TOP 10 cells which receives less voltages :

1. Go to the signoff report directory
2. `cd ~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff`
3. Type the command
4. `tail -n +2 32-irdrop-PVWR.rpt | sort -t',' -k4 -nr | head -10 > top10_vdd.txt`
5. Similarly for vss type the command
6. `tail -n +2 32-irdrop-VGND.rpt | sort -t',' -k4 -nr | head -10 > top10_vss.txt`

```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ column -t -s','
top10_vdd.txt
PHY_47      96.14    73.44    1.79977
PHY_45      96.14    73.44    1.79977
FILLER_0_24_183 84.81    73.44    1.79977
FILLER_0_23_169 84.81    73.44    1.79977
FILLER_0_22_171 84.81    73.44    1.79977
_340_      84.81    73.44    1.79977
_337_      84.81    73.44    1.79977
PHY_43      96.14    68       1.79976
PHY_41      96.14    68       1.79976
hold45      84.81    68       1.79976
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ column -t -s','
top10_vss.txt
PHY_36      5.52     59.84    0.00048724
PHY_34      5.52     59.84    0.00048724
_453_      5.52     59.84    0.00048724
FILLER_0_18_23 12.832   59.84    0.000474945
clkbuf_3_2__f_clk 12.832   59.84    0.000474945
_385_      12.832   59.84    0.000474945
_235_      12.832   59.84    0.000474945
PHY_20      5.52     38.08    0.000422366
PHY_18      5.52     38.08    0.000422366
hold42      5.52     38.08    0.000422366
```



```
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ tail -n +2 32-ir  
drop-VPWR.rpt | sort -t',' -k4 -nr | head -10 > top10_vdd.txt  
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ cat top10_vdd.t  
xt  
PHY_47, 96.14, 73.44, 1.79977  
PHY_45, 96.14, 73.44, 1.79977  
FILLER_0_24_183, 84.81, 73.44, 1.79977  
FILLER_0_23_169, 84.81, 73.44, 1.79977  
FILLER_0_22_171, 84.81, 73.44, 1.79977  
_340_, 84.81, 73.44, 1.79977  
_337_, 84.81, 73.44, 1.79977  
PHY_43, 96.14, 68, 1.79976  
PHY_41, 96.14, 68, 1.79976  
hold45, 84.81, 68, 1.79976  
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ tail -n +2 32-ir  
drop-VGND.rpt | sort -t',' -k4 -nr | head -10 > top10_vss.txt  
bhanu-sharma@bhanu-sharma-VirtualBox:~/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff$ cat top10_vss.t  
xt  
PHY_36, 5.52, 59.84, 0.00048724  
PHY_34, 5.52, 59.84, 0.00048724  
_453_, 5.52, 59.84, 0.00048724  
FILLER_0_18_23, 12.832, 59.84, 0.000474945  
clkbuf_3_2__f_clk, 12.832, 59.84, 0.000474945  
_385_, 12.832, 59.84, 0.000474945  
_235_, 12.832, 59.84, 0.000474945  
PHY_20, 5.52, 38.08, 0.000422366  
PHY_18, 5.52, 38.08, 0.000422366  
hold42, 5.52, 38.08, 0.000422366
```

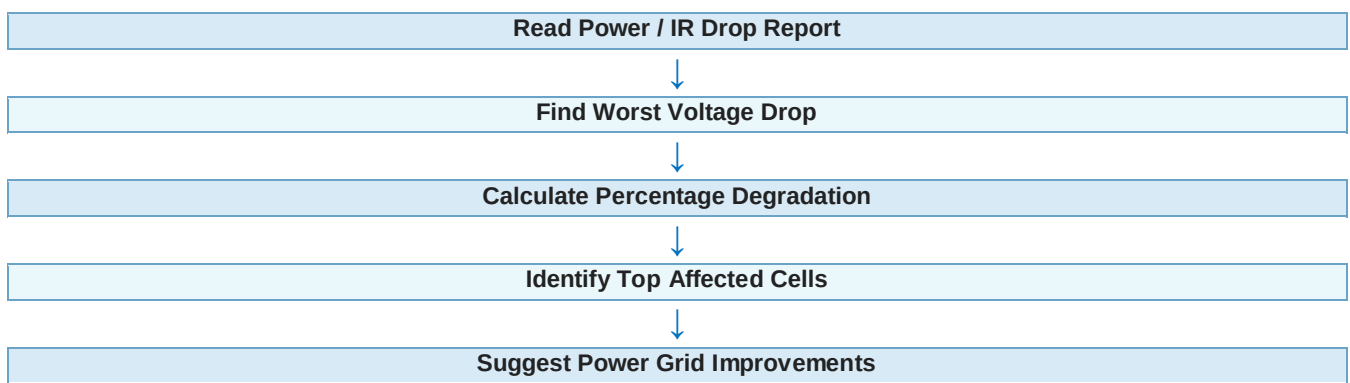


IR Drop Analysis Overview

IR drop analysis checks the voltage drop across the power distribution network of the chip. If some cells receive less voltage than required, timing violations, functional issues, or reliability problems can occur. Therefore, worst IR drop and percentage degradation are important power integrity metrics.

Term	Description
IR Drop	Voltage loss due to resistance in the power network
Worst IR Drop	Maximum observed voltage drop in the design
% Degradation	Percentage reduction from the ideal supply voltage
Affected Cells	Cells receiving lower supply voltage due to IR drop
Improvement Methods	Better power grid, wider metal, more vias, and optimized placement

IR Drop Analysis Flowchart



Tcl script for finding the worst Ir drop :

```

# =====
# IR DROP + % DEGRADATION ANALYSIS
# FOR OPENLANE IR REPORT FORMAT
# =====
# INPUT FILES
# =====
set vdd_file "/home/bhanu-sharma/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff/32-irdrop-VPWR.rpt"
set vss_file "/home/bhanu-sharma/Desktop/Openlane/OpenLane/designs/spm/runs/RUN_1/reports/signoff/32-irdrop-VGND.rpt"
# =====
# SUPPLY VOLTAGE
# =====
set supply_voltage 1.80
# =====
# OUTPUT FILE
# =====
set output_file "final_ir_report.txt"
# =====
# FUNCTION
# =====
proc analyze_ir {file type supply outfp} {
    puts $outfp "\n=====
    puts $outfp "IR ANALYSIS : $type"
    puts $outfp "=====
    if {[file exists $file]} {
        puts $outfp "ERROR : File not found -> $file"
    }
    return
}
set fp [open $file r]
  
```



```
set data {}
while {[gets $fp line] >= 0} {
# Skip header
if {[string match "Instance*" $line]} {
continue
}
if {[string trim $line] eq ""} {
continue
}

# -----
# Split CSV line
# -----
set cols [split $line ","]
if {[llength $cols] < 4} {
continue
}

# -----
# Extract values
# -----
set inst [string trim [lindex $cols 0]]
set xloc [string trim [lindex $cols 1]]
set yloc [string trim [lindex $cols 2]]
set volt [string trim [lindex $cols 3]]
# Numeric check
if {[string is double -strict $volt]} {
continue
}

# -----
# Calculate IR drop
# -----
# IR drop calculation
# -----
if {$type eq "VDD"} {
# VDD drop
set ir_drop [expr {$supply - $volt}]
} else {
# VSS/GND bounce
set ir_drop $volt
}

# Save data
lappend data [list $inst $xloc $yloc $volt $ir_drop]
}

close $fp
# -----
# No data check
# -----
if {[llength $data] == 0} {
puts $outfp "No valid data found."
return
}

# -----
# Sort by IR drop
# -----
set sorted_drop [lsort -real -decreasing -index 4 $data]
set worst [lindex $sorted_drop 0]
set worst_inst [lindex $worst 0]
set worst_x [lindex $worst 1]
set worst_y [lindex $worst 2]
set worst_volt [lindex $worst 3]
set worst_drop [lindex $worst 4]

# -----
# Percentage degradation
# -----
set percent_deg [expr {($worst_drop/$supply)*100.0}]
# -----
# Print worst case
```



```
# -----
puts $outfp "\nWorst IR Drop Cell"
puts $outfp "-----"
puts $outfp "Instance Name      : $worst_inst"
puts $outfp "X Location          : $worst_x"
puts $outfp "Y Location          : $worst_y"
puts $outfp "Received Voltage    : $worst_volt V"
puts $outfp "IR Drop             : [format %.6f $worst_drop] V"
puts $outfp "Percent Degradation : [format %.6f $percent_deg] %"
# -----
# Worst received voltage
# -----
set sorted_voltage [lsort -real -index 3 $data]
set low [lindex $sorted_voltage 0]
puts $outfp "\nWorst Received Voltage"
puts $outfp "-----"
puts $outfp "Instance Name : [lindex $low 0]"
puts $outfp "Voltage       : [lindex $low 3] V"
# -----
# Top 10 Worst Cells
# -----
puts $outfp "\nTop 10 Worst IR Drop Cells"
puts $outfp "-----"
puts $outfp [format "%-25s %-10s %-10s %-15s %-15s" \
"Instance" "X" "Y" "Voltage" "IR_Drop"]
puts $outfp "-----"
set count 0
foreach entry $sorted_drop {
  puts $outfp [format "%-25s %-10s %-10s %-15s %-15.6f" \
[lindex $entry 0] \
[lindex $entry 1] \
[lindex $entry 2] \
[lindex $entry 3] \
[lindex $entry 4]]
incr count
  if {$count >= 10} {
break
}
}

# =====
# MAIN
# =====
set outfp [open $output_file w]
puts $outfp "#####"
puts $outfp "FINAL IR DROP ANALYSIS REPORT"
puts $outfp "#####"
# Analyze VDD
analyze_ir $vdd_file "VDD" $supply_voltage $outfp
# Analyze VSS
analyze_ir $vss_file "VSS" $supply_voltage $outfp
close $outfp
puts "\n=====
puts "FINAL REPORT GENERATED : $output_file"
puts "=====
output :
```



```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane
#####
FINAL IR DROP ANALYSIS REPORT
#####

=====
IR ANALYSIS : VDD
=====

Worst IR Drop Cell
-----
Instance Name      : _229_
X Location         : 5.52
Y Location         : 35.36
Received Voltage   : 1.79947 V
IR Drop            : 0.000530 V
Percent Degradation : 0.029444 %

Worst Received Voltage
-----
Instance Name      : _229_
Voltage            : 1.79947 V

Top 10 Worst IR Drop Cells
-----
Instance           X           Y           Voltage         IR_Drop
-----
_229_              5.52        35.36        1.79947         0.000530
_451_              5.52        35.36        1.79947         0.000530
PHY_16             5.52        35.36        1.79947         0.000530
PHY_18             5.52        35.36        1.79947         0.000530
clkbuf_3_0__f_clk  5.52        35.36        1.79947         0.000530
"final_ir_report.txt" 69 lines, 3271 bytes
```

```
#####
FINAL IR DROP ANALYSIS REPORT
#####

=====
IR ANALYSIS : VDD
=====

Worst IR Drop Cell
-----
Instance Name      : _229_
X Location         : 5.52
Y Location         : 35.36
Received Voltage   : 1.79947 V
IR Drop            : 0.000530 V
Percent Degradation : 0.029444 %
Worst Received Voltage
-----
Instance Name      : _229_
Voltage            : 1.79947 V
Top 10 Worst IR Drop Cells
-----
Instance           X           Y           Voltage         IR_Drop
-----
_229_              5.52        35.36        1.79947         0.000530
_451_              5.52        35.36        1.79947         0.000530
PHY_16             5.52        35.36        1.79947         0.000530
PHY_18             5.52        35.36        1.79947         0.000530
```



clkbuf_3_0__f_clk	5.52	35.36	1.79947	0.000530
225	28.172	40.8	1.79955	0.000450
467	28.172	40.8	1.79955	0.000450
clkbuf_3_1__f_clk	28.172	40.8	1.79955	0.000450
FILLER_0_11_57	28.172	40.8	1.79955	0.000450
FILLER_0_12_53	28.172	40.8	1.79955	0.000450

IR ANALYSIS : VSS

Worst IR Drop Cell

Instance Name : _453_
X Location : 5.52
Y Location : 59.84
Received Voltage : 0.00048724 V
IR Drop : 0.000487 V
Percent Degradation : 0.027069 %
Worst Received Voltage

Instance Name : _406_
Voltage : 0.000231324 V
Top 10 Worst IR Drop Cells

Instance	X	Y	Voltage	IR_Drop
-----	-----	-----	-----	-----
453	5.52	59.84	0.00048724	0.000487
PHY_34	5.52	59.84	0.00048724	0.000487
PHY_36	5.52	59.84	0.00048724	0.000487
235	12.832	59.84	0.000474945	0.000475
385	12.832	59.84	0.000474945	0.000475
clkbuf_3_2__f_clk	12.832	59.84	0.000474945	0.000475
FILLER_0_18_23	12.832	59.84	0.000474945	0.000475
229	5.52	38.08	0.000422366	0.000422
PHY_18	5.52	38.08	0.000422366	0.000422
PHY_20	5.52	38.08	0.000422366	0.000422

Levels Of Logic :



Design Analysis Using STA Timing Report

The Levels of Logic (LOL) and slack analysis were performed on the post-route Static Timing Analysis (STA) report generated during the OpenLane physical design flow. A standalone TCL script was developed to automatically parse the timing report and extract the logic depth and slack information for all valid timing paths.

Timing report used : 31-rcx_sta.max.rpt

Levels of Logic represent the number of logic stages encountered between the startpoint and endpoint of a timing path.

A timing path generally consists of:

Startpoint → Combinational Logic → Endpoint

Higher LOL generally indicates:

- Larger combinational depth
- Higher propagation delay
- Increased timing criticality

TCL script for Finding the LOL :

```
# =====
# Levels_Of_Logic.tcl
# LOL + Slack Extraction from STA Report
# =====

# Input STA report
set sta_file "31-rcx_sta.max.rpt"

# Output report
set out_file "logic_level_report.txt"

# Open files
set fp [open $sta_file r]
set out [open $out_file w]

# Variables
set path_num 0
set logic_count 0
set valid_path 0

set current_slack "N/A"

set max_lol 0
set min_lol 999999

set worst_slack 999999
set best_slack -999999

set total_paths 0

puts $out "=====
puts $out " LEVELS OF LOGIC + SLACK REPORT "
puts $out "=====
```



```
while {[gets $fp line] >= 0} {

# -----
# Detect Startpoint
# -----
if {[string match "*Startpoint:*" $line]} {

# Save previous path result
if {$valid_path && $path_num > 0} {

puts $out "Path $path_num : LOL = $logic_count | Slack = $current_slack"

incr total_paths

# Update MAX LOL
if {$logic_count > $max_lol} {
set max_lol $logic_count
}

# Update MIN LOL
if {$logic_count < $min_lol} {
set min_lol $logic_count
}

# Update slack values
if {$current_slack != "N/A"} {

if {$current_slack < $worst_slack} {
set worst_slack $current_slack
}

if {$current_slack > $best_slack} {
set best_slack $current_slack
}
}
}

incr path_num
set logic_count 0
set current_slack "N/A"
set valid_path 1

puts $out "\n-----"
puts $out "$line"
}

# -----
# Endpoint
# -----
if {[string match "*Endpoint:*" $line]} {

puts $out "$line"
```



```
# Ignore recovery paths
if {[string match "*recovery*" $line]} {
    set valid_path 0
}
}

# -----
# Count Logic Levels
# -----
if {$valid_path} {

    if {[regexp {[A-Za-z0-9_]+}/([A-Za-z0-9_]+)} $line match]} {

        if {
            ![string match "(net)*" $line] &&
            ![string match "arrival time*" $line] &&
            ![string match "required time*" $line] &&
            ![string match "slack*" $line]
        } {

            incr logic_count
        }
    }
}

# -----
# Extract Slack
# -----
if {[regexp {[(-0-9\.)+}\s+slack} $line match slack]} {

    set current_slack $slack
}
}

# -----
# Final Path
# -----
if {$valid_path && $path_num > 0} {

    puts $out "Path $path_num : LOL = $logic_count | Slack = $current_slack"

    incr total_paths

    if {$logic_count > $max_lol} {
        set max_lol $logic_count
    }

    if {$logic_count < $min_lol} {
        set min_lol $logic_count
    }

    if {$current_slack != "N/A"} {

        if {$current_slack < $worst_slack} {
```



```
set worst_slack $current_slack
}

if {$current_slack > $best_slack} {
    set best_slack $current_slack
}
}
}

# -----
# Final Summary
# -----

puts $out "\n=====
puts $out " FINAL SUMMARY "
puts $out "=====

puts $out "Total Timing Paths Analyzed : $total_paths"
puts $out "Maximum Levels of Logic    : $max_lol"
puts $out "Minimum Levels of Logic    : $min_lol"
puts $out "Worst Slack                : $worst_slack"
puts $out "Best Slack                 : $best_slack"

close $fp
close $out

puts "\n=====
puts "Logic Level + Slack Report Generated Successfully"
puts "Output File : $out_file"
puts "=====
```

Output :

```
=====
LEVELS OF LOGIC
=====
```

```
-----
Startpoint: _444_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: p (output port clocked by clk)
Path 1 : LOL = 8 | Slack = 6.79
```

```
-----
Startpoint: y (input port clocked by clk)
Endpoint: _503_ (rising edge-triggered flip-flop clocked by clk)
Path 2 : LOL = 16 | Slack = 7.02
```

```
-----
Startpoint: y (input port clocked by clk)
Endpoint: _487_ (rising edge-triggered flip-flop clocked by clk)
Path 3 : LOL = 16 | Slack = 7.03
```



Startpoint: y (input port clocked by clk)
Endpoint: _501_ (rising edge-triggered flip-flop clocked by clk)
Path 4 : LOL = 16 | Slack = 7.04

Startpoint: y (input port clocked by clk)
Endpoint: _495_ (rising edge-triggered flip-flop clocked by clk)
Path 5 : LOL = 16 | Slack = 7.04

Startpoint: y (input port clocked by clk)
Endpoint: _497_ (rising edge-triggered flip-flop clocked by clk)
Path 6 : LOL = 16 | Slack = 7.04

Startpoint: y (input port clocked by clk)
Endpoint: _469_ (rising edge-triggered flip-flop clocked by clk)
Path 7 : LOL = 16 | Slack = 7.04

Startpoint: y (input port clocked by clk)
Endpoint: _473_ (rising edge-triggered flip-flop clocked by clk)
Path 8 : LOL = 16 | Slack = 7.04

Startpoint: y (input port clocked by clk)
Endpoint: _485_ (rising edge-triggered flip-flop clocked by clk)
Path 9 : LOL = 16 | Slack = 7.04

Startpoint: y (input port clocked by clk)
Endpoint: _499_ (rising edge-triggered flip-flop clocked by clk)
Path 10 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)
Endpoint: _471_ (rising edge-triggered flip-flop clocked by clk)
Path 11 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)
Endpoint: _481_ (rising edge-triggered flip-flop clocked by clk)
Path 12 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)
Endpoint: _479_ (rising edge-triggered flip-flop clocked by clk)
Path 13 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)
Endpoint: _493_ (rising edge-triggered flip-flop clocked by clk)



Path 14 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _489_ (rising edge-triggered flip-flop clocked by clk)

Path 15 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _467_ (rising edge-triggered flip-flop clocked by clk)

Path 16 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _465_ (rising edge-triggered flip-flop clocked by clk)

Path 17 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _491_ (rising edge-triggered flip-flop clocked by clk)

Path 18 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _475_ (rising edge-triggered flip-flop clocked by clk)

Path 19 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _451_ (rising edge-triggered flip-flop clocked by clk)

Path 20 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _477_ (rising edge-triggered flip-flop clocked by clk)

Path 21 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _459_ (rising edge-triggered flip-flop clocked by clk)

Path 22 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _483_ (rising edge-triggered flip-flop clocked by clk)

Path 23 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _443_ (rising edge-triggered flip-flop clocked by clk)

Path 24 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)



Endpoint: _461_ (rising edge-triggered flip-flop clocked by clk)

Path 25 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _457_ (rising edge-triggered flip-flop clocked by clk)

Path 26 : LOL = 16 | Slack = 7.05

Startpoint: y (input port clocked by clk)

Endpoint: _449_ (rising edge-triggered flip-flop clocked by clk)

Path 27 : LOL = 16 | Slack = 7.06

Startpoint: y (input port clocked by clk)

Endpoint: _463_ (rising edge-triggered flip-flop clocked by clk)

Path 28 : LOL = 16 | Slack = 7.06

Startpoint: y (input port clocked by clk)

Endpoint: _453_ (rising edge-triggered flip-flop clocked by clk)

Path 29 : LOL = 16 | Slack = 7.06

Startpoint: y (input port clocked by clk)

Endpoint: _447_ (rising edge-triggered flip-flop clocked by clk)

Path 30 : LOL = 16 | Slack = 7.06

Startpoint: y (input port clocked by clk)

Endpoint: _455_ (rising edge-triggered flip-flop clocked by clk)

Path 31 : LOL = 16 | Slack = 7.06

Startpoint: y (input port clocked by clk)

Endpoint: _450_ (rising edge-triggered flip-flop clocked by clk)

Path 32 : LOL = 16 | Slack = 7.18

Startpoint: y (input port clocked by clk)

Endpoint: _474_ (rising edge-triggered flip-flop clocked by clk)

Path 33 : LOL = 16 | Slack = 7.19

Startpoint: y (input port clocked by clk)

Endpoint: _470_ (rising edge-triggered flip-flop clocked by clk)

Path 34 : LOL = 16 | Slack = 7.19

Startpoint: y (input port clocked by clk)

Endpoint: _454_ (rising edge-triggered flip-flop clocked by clk)

Path 35 : LOL = 16 | Slack = 7.20



Startpoint: y (input port clocked by clk)
Endpoint: _444_ (rising edge-triggered flip-flop clocked by clk)
Path 36 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _452_ (rising edge-triggered flip-flop clocked by clk)
Path 37 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _448_ (rising edge-triggered flip-flop clocked by clk)
Path 38 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _456_ (rising edge-triggered flip-flop clocked by clk)
Path 39 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _458_ (rising edge-triggered flip-flop clocked by clk)
Path 40 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _472_ (rising edge-triggered flip-flop clocked by clk)
Path 41 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _482_ (rising edge-triggered flip-flop clocked by clk)
Path 42 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _462_ (rising edge-triggered flip-flop clocked by clk)
Path 43 : LOL = 16 | Slack = 7.20

Startpoint: y (input port clocked by clk)
Endpoint: _484_ (rising edge-triggered flip-flop clocked by clk)
Path 44 : LOL = 16 | Slack = 7.21

Startpoint: y (input port clocked by clk)
Endpoint: _476_ (rising edge-triggered flip-flop clocked by clk)
Path 45 : LOL = 16 | Slack = 7.21

Startpoint: y (input port clocked by clk)
Endpoint: _494_ (rising edge-triggered flip-flop clocked by clk)
Path 46 : LOL = 16 | Slack = 7.21



Startpoint: y (input port clocked by clk)
Endpoint: _478_ (rising edge-triggered flip-flop clocked by clk)
Path 47 : LOL = 16 | Slack = 7.21

Startpoint: y (input port clocked by clk)
Endpoint: _480_ (rising edge-triggered flip-flop clocked by clk)
Path 48 : LOL = 16 | Slack = 7.21

Startpoint: y (input port clocked by clk)
Endpoint: _496_ (rising edge-triggered flip-flop clocked by clk)
Path 49 : LOL = 16 | Slack = 7.21

Startpoint: y (input port clocked by clk)
Endpoint: _468_ (rising edge-triggered flip-flop clocked by clk)
Path 50 : LOL = 16 | Slack = 7.22

Startpoint: y (input port clocked by clk)
Endpoint: _466_ (rising edge-triggered flip-flop clocked by clk)
Path 51 : LOL = 16 | Slack = 7.22

Startpoint: y (input port clocked by clk)
Endpoint: _490_ (rising edge-triggered flip-flop clocked by clk)
Path 52 : LOL = 16 | Slack = 7.22

Startpoint: y (input port clocked by clk)
Endpoint: _492_ (rising edge-triggered flip-flop clocked by clk)
Path 53 : LOL = 16 | Slack = 7.22

Startpoint: y (input port clocked by clk)
Endpoint: _486_ (rising edge-triggered flip-flop clocked by clk)
Path 54 : LOL = 16 | Slack = 7.22

Startpoint: y (input port clocked by clk)
Endpoint: _460_ (rising edge-triggered flip-flop clocked by clk)
Path 55 : LOL = 16 | Slack = 7.23

Startpoint: y (input port clocked by clk)
Endpoint: _464_ (rising edge-triggered flip-flop clocked by clk)
Path 56 : LOL = 16 | Slack = 7.23

Startpoint: y (input port clocked by clk)
Endpoint: _488_ (rising edge-triggered flip-flop clocked by clk)
Path 57 : LOL = 16 | Slack = 7.23



Startpoint: y (input port clocked by clk)
Endpoint: _505_ (rising edge-triggered flip-flop clocked by clk)
Path 58 : LOL = 14 | Slack = 7.29

Startpoint: y (input port clocked by clk)
Endpoint: _446_ (rising edge-triggered flip-flop clocked by clk)
Path 59 : LOL = 16 | Slack = 7.31

Startpoint: y (input port clocked by clk)
Endpoint: _506_ (rising edge-triggered flip-flop clocked by clk)
Path 60 : LOL = 14 | Slack = 7.45

Startpoint: y (input port clocked by clk)
Endpoint: _504_ (rising edge-triggered flip-flop clocked by clk)
Path 61 : LOL = 14 | Slack = 7.46

Startpoint: y (input port clocked by clk)
Endpoint: _502_ (rising edge-triggered flip-flop clocked by clk)
Path 62 : LOL = 14 | Slack = 7.46

Startpoint: y (input port clocked by clk)
Endpoint: _500_ (rising edge-triggered flip-flop clocked by clk)
Path 63 : LOL = 14 | Slack = 7.47

Startpoint: y (input port clocked by clk)
Endpoint: _498_ (rising edge-triggered flip-flop clocked by clk)
Path 64 : LOL = 14 | Slack = 7.47

Startpoint: y (input port clocked by clk)
Endpoint: _445_ (rising edge-triggered flip-flop clocked by clk)
Path 65 : LOL = 12 | Slack = 7.52

Startpoint: rst (input port clocked by clk)
Endpoint: _473_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _478_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _465_ (recovery check against rising-edge clock clk)



Startpoint: rst (input port clocked by clk)

Endpoint: _481_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _475_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _476_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _474_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _477_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _482_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _479_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _471_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _480_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _469_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _468_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _472_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _450_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _500_ (recovery check against rising-edge clock clk)



Startpoint: rst (input port clocked by clk)
Endpoint: _487_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _470_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _499_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _502_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _497_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _483_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _498_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _501_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _495_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _467_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _446_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _459_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _445_ (recovery check against rising-edge clock clk)



Startpoint: rst (input port clocked by clk)

Endpoint: _458_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _454_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _456_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _452_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _461_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _493_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _484_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _496_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _457_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _462_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _453_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _460_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _455_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _494_ (recovery check against rising-edge clock clk)



Startpoint: rst (input port clocked by clk)
Endpoint: _449_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _451_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _444_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _443_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _447_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _448_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _464_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _492_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _485_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _490_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _466_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _488_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)
Endpoint: _489_ (recovery check against rising-edge clock clk)



Startpoint: rst (input port clocked by clk)

Endpoint: _491_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _463_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _486_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _505_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _503_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _506_ (recovery check against rising-edge clock clk)

Startpoint: rst (input port clocked by clk)

Endpoint: _504_ (recovery check against rising-edge clock clk)

FINAL SUMMARY

Total Timing Paths Analyzed : 65

Maximum Levels of Logic : 16

Minimum Levels of Logic : 8

Worst Slack : 6.79

Best Slack : 7.52

Path No.	Startpoint	Endpoint	LOL	Slack (ns)
1	_444_	p	8	6.79
2	y	_503_	16	7.02
10	y	_499_	16	7.05
32	y	_450_	16	7.18
65	y	_445_	12	7.52



```
bhanu-sharma@bhanu-sharma-VirtualBox: ~/Desktop/Openlane/OpenLane

Startpoint: rst (input port clocked by clk)
Endpoint: _463_ (recovery check against rising-edge clock clk)

-----
Startpoint: rst (input port clocked by clk)
Endpoint: _486_ (recovery check against rising-edge clock clk)

-----
Startpoint: rst (input port clocked by clk)
Endpoint: _505_ (recovery check against rising-edge clock clk)

-----
Startpoint: rst (input port clocked by clk)
Endpoint: _503_ (recovery check against rising-edge clock clk)

-----
Startpoint: rst (input port clocked by clk)
Endpoint: _506_ (recovery check against rising-edge clock clk)

-----
Startpoint: rst (input port clocked by clk)
Endpoint: _504_ (recovery check against rising-edge clock clk)

=====
FINAL SUMMARY
=====
Total Timing Paths Analyzed : 65
Maximum Levels of Logic      : 16
Minimum Levels of Logic      : 8
Worst Slack                   : 6.79
```

Conclusion

In this assignment, basic Linux commands and their practical use in the VLSI design environment were studied. Commands related to file handling, directory management, text processing, permission control, and editing were executed and verified using terminal screenshots.

The OpenLane physical design environment was also explored. The assignment covered OpenLane architecture, directory structure, report analysis, log analysis, wire length extraction, and IR drop analysis. TCL scripting was used to automate the extraction of design metrics from OpenLane routing logs.

Overall, this assignment helped in understanding how Linux commands and OpenLane tools are used together in the physical design flow. It also provided practical exposure to report analysis and automation, which are important skills in VLSI physical design.

Learning Area	Outcome
Linux Commands	Understood essential terminal commands used in VLSI workflows
OpenLane Flow	Explored RTL-to-GDSII physical design stages
Reports and Logs	Learned how to analyze outputs, warnings, and design metrics
TCL Scripting	Used automation to extract wire length and IR drop information